

The AI Ritual Concierge, Explained Simply

What I built for Maison Hygia's AI & Innovation case study, and why

What Am I Actually Building?

Imagine you're a Maison Hygia customer and you type this into a chat box:

"My skin feels dry lately, I'm not sleeping particularly well, and I want to create a better evening routine without making it complicated."

A normal website makes you scroll through dozens of products and guess which ones matter. What I built instead is a small program that reads a message like that, figures out what you actually need, picks two or three products that make sense together, and explains why it picked them — in plain English, without pretending to be a doctor.

I call it the AI Ritual Concierge. It's a prototype, not a finished product, but every piece of it actually runs — it's not just a diagram or a mockup.

How It Works, Step by Step

There are four steps, and I tried to make each one as easy to explain out loud as possible.

1. Understand

It reads your message and picks out the important bits: what's bothering you (dry skin, bad sleep, low energy...), whether this is a morning or evening thing, and whether you asked for something simple. It does this by spotting keywords — nothing fancy.

2. Retrieve

It compares your message to every product's description and counts how many words they have in common. More matching words means the product is probably more relevant to you. This is a simple version of something real AI systems call "RAG" — which stands for Retrieval-Augmented Generation. The fancy version uses AI to understand meaning, not just matching words, but the idea is the same: go find real information before you answer, instead of just guessing.

3. Recommend

It picks two or three products for you, and it scores them mostly by how relevant they are to your actual problem. How profitable a product is for Maison Hygia is allowed to nudge the ranking a little, but never enough to override relevance. This part matters more than it sounds like it should — more on that below.

4. Explain

For each product, it tells you plainly why it was picked: which concern it addresses, which ingredient does the work, and what that ingredient is generally known for. It never claims to treat or cure anything — it sticks to wellness language, on purpose.

The Most Important Decision I Made

Here's a problem that's easy to miss until it's already happened: what if the system quietly started recommending the most expensive products, because those make the company more money? It would still “work,” technically — but it would stop actually helping the customer, and the first person who notices gets suspicious of every recommendation after that. That's a real way to lose someone's trust for good.

So I built the scoring on purpose to prevent that. Relevance to your actual problem counts for 85% of the score. How profitable the product is only counts for 15%, so it can break a tie between two similarly good options, but it can never win on its own.

Here's what that looks like with real numbers from the prototype:

What I shipped (fair)	What I rejected (profit-first)
Golden Hour Vitamin C Serum — relevance 2, \$48	Golden Hour Vitamin C Serum — relevance 2, \$48
Bright Start Electrolyte Blend — relevance 2, \$22	Moonpetal Overnight Balm — relevance 1, \$42 (a face balm, swapped in even though it has nothing to do with low energy)

Same customer, same products available — but the profit-first version swaps in a \$42 face balm that has nothing to do with feeling tired, just because it's more profitable than the electrolyte blend that actually fits. That's the exact mistake the case study asks candidates to design against, and now I have real, runnable proof that my scoring formula avoids it.

What's Actually in the Code (No Jargon)

- `products.json` — 8 made-up Maison Hygia products, each with a name, price, key ingredient, and which problems it helps with.
- `concierge.py` — the whole program, written in plain Python with no extra libraries to install. You can read it top to bottom like a recipe: understand, retrieve, recommend, explain.
- `README.md` — a plain-language walkthrough of every step, plus an honest list of what this prototype doesn't do yet.

You can run the whole thing yourself with one line, no setup: `python concierge.py`

What I'd Build Next

I want to be upfront about where this prototype is simplified, because I think that's more convincing than pretending it's finished:

- Right now it matches products by counting shared words. A real version would use an AI embedding model, so it understands that “tired” and “low energy” mean the same thing even without an exact word match.
- The “understand your message” step uses a fixed list of keywords. A real version would use an LLM like Claude, so it generalizes to messages I didn't think to anticipate.
- It doesn't remember you between conversations yet. A real version would keep a lightweight profile of your preferences so the ritual gets better over time.
- There's no automatic safety check on the generated explanations yet — right now that's handled by keeping the explanation template simple and controlled.

THANK YOU